Ansible Question Index - All 75 Questions

Q#	Question
Q1	What is configuration management? Why not shell scripts?
Q2	Ansible architecture - agentless, SSH, push-based
Q3	Ansible.cfg - configuration file, precedence, settings
Q4	Inventory - static, dynamic, groups, host/group variables
Q5	Ad-hoc commands - one-off tasks without playbooks
Q6	Playbooks - YAML structure, plays, tasks, handlers, become
Q7	Modules - command, shell, copy, template, file, service, package
Q8	Variables - host_vars, group_vars, extra_vars, register, set_fact
Q9	Facts - gathering system info, custom facts, fact caching
Q10	Conditionals - when, changed_when, failed_when
Q11	Loops - loop, with_items, with_dict, with_fileglob
Q12	Handlers and notify - triggering actions on change
Q13	Templates - Jinja2, substitution, conditionals, loops
Q14	Ansible Vault - encrypting secrets in playbooks
Q15	Roles - directory structure, defaults, tasks, handlers
Q16	Galaxy - collections, community roles, requirements.yml
Q17	Tags - running/skipping specific tasks
Q18	Idempotency - why it matters, non-idempotent modules
Q19	Connection plugins - SSH, local, docker, WinRM
Q20	Privilege escalation - become, become_user, become_method
Q21	Check mode and diff mode - dry run, safe operations
Q22	Ansible vs Puppet vs Chef vs SaltStack

Q23	Dynamic inventory - AWS, GCP, Azure, inventory plugins
Q24	Collections - namespaces, installation, requirements
Q25	Tower / AWX - GUI, RBAC, scheduling, workflows
Q26	Optimization - pipelining, mitogen, async, serial, forks
Q27	Error handling - block/rescue/always, ignore_errors
Q28	Include vs import - static vs dynamic inclusion
Q29	Delegation - delegate_to, local_action, run_once
Q30	Rolling deployments - serial, max_fail_percentage
Q31	Magic variables - inventory_hostname, hostvars, groups
Q32	Strategy plugins - linear, free, debug, host_pinned
Q33	Ansible with Docker and Kubernetes
Q34	Ansible in CI/CD - Jenkins, GitHub Actions, GitLab
Q35	Testing - Molecule, ansible-lint, yamllint
Q36	Variable precedence - the 22 levels of priority
Q37	Lookups and filters - file, env, pipe, custom filters
Q38	Callback plugins - output, logging, profiling
Q39	Ansible and Terraform - when to use which
Q40	Secrets management - Vault integration, no_log
Q41	Multi-environment - inventory per env, group_vars
Q42	Windows management - WinRM, PowerShell modules
Q43	Network automation - ios_config, nxos, Arista
Q44	Custom modules - writing Python modules
Q45	Performance at scale - SSH multiplexing, fact caching
Q46	Asynchronous actions - async, poll, fire-and-forget

Q47	Architecture at scale - 1000+ hosts, AWX clustering
Q48	Execution environments - containerized, ansible-builder
Q49	Ansible Automation Platform - controller, hub, mesh
Q50	Event-Driven Ansible - rulebooks, event sources
Q51	GitOps with Ansible - pull-based, drift detection
Q52	Immutable infra vs config management - when Ansible is wrong
Q53	Security hardening - escalation, SSH, audit
Q54	Migration from Puppet/Chef - strategy, pitfalls
Q55	Infrastructure testing - InSpec, ServerSpec, Goss
Q56	Playbook works on 1 host fails on 50
Q57	Task not idempotent - runs every time
Q58	Variable precedence - wrong value in production
Q59	Vault password lost - secrets inaccessible
Q60	Playbook ran against wrong inventory - hit production
Q61	Rolling deployment failed midway
Q62	Template rendered incorrectly - Jinja2 not expanded
Q63	Ansible taking 30 min for 100 hosts - optimize
Q64	Handler not running - notify name mismatch
Q65	Secret leaked in output - no_log missing
Q66	Dynamic inventory returning stale hosts
Q67	Configuration drift - servers don't match playbook
Q68	SSH host key checking failed on new servers
Q69	Cannot find Python on target host
Q70	Playbook hangs forever - SSH timeout

Q71	become password required but not provided
Q72	File permissions changed on every run
Q73	include_role variables leaking between hosts
Q74	Galaxy role broke after update
Q75	Write a playbook: deploy Nginx with zero downtime

SECTION 1 - Junior Core Concepts (Q1-Q22)

Q1 What is configuration management? Why not shell scripts?

The Simple Answer

Configuration management (CM) automates the setup, maintenance, and enforcement of consistent system configurations across servers. Instead of manually configuring each server or writing brittle shell scripts, you declare the desired state and the CM tool ensures every server matches it.

The Analogy - Building Standards

> Shell scripts are like giving each construction worker verbal instructions - each interprets them differently, mistakes happen, and there is no guarantee two buildings end up the same. Configuration management is like giving them detailed blueprints and building codes - every building follows the same standard, inspectors verify compliance, and deviations are automatically corrected.

Why Shell Scripts Fail at Scale

Not idempotent - Running a shell script twice may break things (adding duplicate entries, restarting services unnecessarily). CM tools check current state before acting - if the package is already installed, they skip it.

No state awareness - Shell scripts do not know what has already been done. They execute blindly from top to bottom. CM tools compare desired state with actual state and only make necessary changes.

Poor error handling - A failed command in a script requires complex error handling code. CM tools have built-in retry, rescue blocks, and reporting.

No inventory management - Shell scripts require manual lists of servers or wrapper scripts for SSH loops. CM tools have sophisticated inventory systems with grouping, variables, and dynamic discovery.

No audit trail - Shell scripts leave no record of what changed. CM tools log every change with before/after values.

> **When shell scripts ARE appropriate:** Simple bootstrapping (cloud-init user data), one-off emergency fixes, tasks that will never be repeated, and glue scripts that call Ansible. If the task is under 20 lines and runs once, a shell script is fine.

Q2 Ansible architecture - agentless, SSH, push-based

The Simple Answer

Ansible uses an agentless, push-based architecture. The control node (your laptop or CI server) connects to managed nodes via SSH, pushes Python modules, executes them, and collects results. No software needs to be installed on managed nodes beyond Python and SSH - both are pre-installed on virtually every Linux server.

The Analogy - A Chef vs a Kitchen Robot

> Puppet and Chef install an agent (kitchen robot) on every server that continuously checks for new instructions. Ansible is like a chef who walks into each kitchen (SSH), cooks the meal (runs modules), and leaves. No permanent presence on the server. The trade-off: Ansible only enforces state when you run it (push). Puppet/Chef continuously enforce state (pull).

Components

Control node - Where Ansible runs. Your laptop, a CI runner, or an AWX/Tower server. Must be Linux/macOS (not Windows natively - use WSL).

Managed nodes - The servers being configured. Need Python 3 and SSH. No Ansible installation required.

Inventory - List of managed nodes organized into groups.

Playbook - YAML file defining the desired state. Contains plays, tasks, handlers.

Modules - Units of work: install a package (apt/yum), copy a file (copy), manage a service (service), create a user (user). 3000+ built-in modules.

Plugins - Extend Ansible functionality: connection plugins (SSH, WinRM), lookup plugins (read from files/Vault), callback plugins (custom output), filter plugins (data transformation).

> **Why agentless matters in interviews:** No agent to install, update, secure, or troubleshoot on every server. No agent consuming resources. No agent port to open in firewalls. Reduced attack surface. The trade-off: SSH must be available and Python must be installed on targets.

Q3 Ansible.cfg - configuration file, precedence, settings

The Simple Answer

ansible.cfg configures Ansible's behavior - where to find inventory, how to connect, performance settings, and privilege escalation defaults. Ansible searches for the config file in this order (first found wins): ANSIBLE_CONFIG environment variable → ./ansible.cfg (current directory) → ~/.ansible.cfg (home directory) → /etc/ansible/ansible.cfg (system-wide).

Critical Settings

[defaults] - inventory (path to inventory file), remote_user (SSH user), forks (parallel connections, default 5 - increase to 20-50 for scale), host_key_checking (disable for dynamic cloud environments, but security trade-off), timeout (SSH connection timeout), roles_path (where to find roles).

[privilege_escalation] - become (True/False), become_method (sudo), become_user (root), become_ask_pass (prompt for sudo password).

[ssh_connection] - pipelining (True - dramatically speeds up execution by reducing SSH operations), ssh_args (custom SSH arguments like ControlPersist for connection reuse).

> **The interview question:** "I'm running Ansible but it's using the wrong settings. How do I find which config file it's loading?" Run ansible --version - it shows the config file path. Or set ANSIBLE_CONFIG explicitly to force a specific file.

Q4 Inventory - static, dynamic, groups, variables

The Simple Answer

The inventory defines which hosts Ansible manages and how they are organized. It can be a simple INI or YAML file (static) or a script/plugin that queries cloud APIs (dynamic).

Static Inventory (INI format)

```
[webservers]

web1.example.com

web2.example.com ansible_host=10.0.1.5
```

[databases]

db1.example.com ansible_port=2222

[production:children]

webservers

databases

[webservers:vars]

http_port=80

```

### Key Concepts

**Groups** - Organize hosts by function (webservers, databases), environment (production, staging), or region. The all group contains every host. The ungrouped group contains hosts not in any other group.

**Host variables** - Per-host configuration in the inventory or in host_vars/hostname.yml files. Override group variables.

**Group variables** - Per-group configuration in the inventory or in group_vars/groupname.yml files. Applied to all hosts in the group.

**Children groups** - Groups containing other groups. production:children includes webservers and databases.

> **Best practice:** Use YAML inventory for complex setups. Store host_vars/ and group_vars/ as separate files alongside the inventory. Keep secrets in Ansible Vault-encrypted files within group_vars/.

## Q5 Ad-hoc commands - one-off tasks

### The Simple Answer

Ad-hoc commands run a single Ansible module against hosts without writing a playbook. Used for quick tasks, troubleshooting, and gathering information.

### Examples

ansible all -m ping - Test connectivity to all hosts. ansible webservers -m shell -a "df -h" - Check disk space. ansible databases -m service -a "name=postgresql state=restarted" -b - Restart PostgreSQL with sudo. ansible all -m setup - Gather all facts from all hosts. ansible webservers -m copy -a "src=./app.conf dest=/etc/app/app.conf" -b - Copy a config file.

### When to Use Ad-hoc vs Playbooks

Ad-hoc: quick checks, one-time tasks, troubleshooting. Playbooks: repeatable configurations, multi-step operations, anything that should be version-controlled.

## Q6 Playbooks - YAML structure, plays, tasks, handlers

### The Simple Answer

A playbook is a YAML file containing one or more plays. Each play targets a group of hosts and defines tasks to execute. Tasks call modules. Handlers are special tasks that run only when notified by another task (typically for service restarts).

### Playbook Structure
```

A play has: name (description), hosts (target hosts/groups), become (privilege escalation), vars (variables), tasks (list of module calls), and handlers (triggered by notify).

Task Anatomy

Each task has: name (human-readable description), module name with parameters, and optional: when (conditional), register (capture output), notify (trigger handler), tags, become.

Handlers

Handlers run at the end of a play, only if notified by a changed task. Common pattern: a template task copies a config file and notifies a restart handler. If the config did not change, the handler does not run. If 10 tasks notify the same handler, it runs only once. This prevents unnecessary service restarts.

> **The key insight:** Tasks run in order, every time. Handlers run at the end, only when notified, only once. This makes playbooks efficient - config file changes trigger a restart only if the file actually changed.

Q7 Modules - command, shell, copy, template, file, service, package

The Simple Answer

Modules are the building blocks of Ansible. Each module handles a specific type of task.

package - Install/remove packages. Works across apt, yum, dnf automatically: package: name=nginx state=present.

service - Manage services: service: name=nginx state=started enabled=yes.

copy - Copy files from control node to managed node: copy: src=app.conf dest=/etc/app/app.conf owner=root mode=0644.

template - Like copy but processes Jinja2 templates first. Variables are substituted before the file is placed on the target.

file - Manage file properties: file: path=/opt/app state=directory owner=app mode=0755.

command - Run a command (no shell features like pipes, redirects). Not idempotent by default - use creates: or removes: to add idempotency.

shell - Run a command through the shell (supports pipes, redirects). Not idempotent. Avoid when a specific module exists.

user - Manage users: user: name=deploy groups=docker shell=/bin/bash.

lineinfile - Ensure a specific line exists in a file. Idempotent.

> **The golden rule:** Always use a specific module instead of command/shell. Modules are idempotent, provide useful changed/ok status, and handle edge cases. command and shell are escape hatches for when no module exists - they should be rare in production playbooks.

Q8 Variables - host_vars, group_vars, extra_vars, register, set_fact

The Simple Answer

Variables customize playbook behavior across different hosts and environments. The same playbook deploys to dev and prod using different variables for database URLs, replica counts, and feature flags.

Variable Sources

group_vars/groupname.yml - Variables applied to all hosts in a group. group_vars/production.yml applies to all production hosts.

host_vars/hostname.yml - Variables for a specific host. Override group variables.

Playbook vars - Defined in the play with vars: or vars_files:.

extra_vars (-e) - Passed on the command line. Highest precedence. ansible-playbook deploy.yml -e "version=2.0.1". Used for CI/CD where the version comes from the pipeline.

register - Capture a task's output into a variable: register: result. Access with result.stdout, result.rc, result.changed.

set_fact - Create a variable dynamically during playbook execution: set_fact: app_url: "https://{{ domain }}/{{ path }}". Unlike register (which captures module output), set_fact creates arbitrary variables from expressions.

> **register vs set_fact:** Use register when you need the output of a command (stdout, return code). Use set_fact when you need to compute a new variable from existing variables.

Q9 Facts - gathering system info, custom facts, caching

The Simple Answer

Facts are automatically gathered information about managed nodes - OS, IP addresses, CPU count, memory, disk, network interfaces. Collected by the setup module at the start of every play (unless disabled with gather_facts: false).

Accessing Facts

ansible_os_family (Debian, RedHat), ansible_distribution (Ubuntu, CentOS), ansible_default_ipv4.address (primary IP), ansible_memtotal_mb (total RAM), ansible_processor_vcpus (CPU count). Use in conditionals: when: ansible_os_family == "Debian" to run apt-specific tasks only on Debian-based systems.

Custom Facts

Place JSON or INI files in /etc/ansible/facts.d/ on managed nodes. Ansible collects them as ansible_local variables. Useful for: application version tracking, server role identification, and custom metadata.

Fact Caching

Gathering facts from 500 hosts takes time. Enable fact caching to store facts between runs: fact_caching = jsonfile, fact_caching_connection = /tmp/facts_cache, fact_caching_timeout = 86400. Facts are cached for 24 hours. Subsequent playbook runs skip fact gathering.

Q10 Conditionals - when, changed_when, failed_when

The Simple Answer

when - Skip a task unless a condition is true. when: ansible_os_family == "Debian" runs the task only on Debian/Ubuntu. when: result.rc != 0 runs only if a previous command failed. Conditions use Jinja2 expressions - no {{ }} needed (when already evaluates as Jinja2).

changed_when - Override when Ansible considers a task "changed." By default, command/shell tasks are always "changed." Set changed_when: result.stdout != "" to only report changed when there is output - prevents unnecessary handler triggers.

failed_when - Override when Ansible considers a task "failed." Some commands return non-zero exit codes for normal conditions. failed_when: result.rc != 0 and 'already exists' not in result.stderr - do not fail if the error is expected.

Q11 Loops - loop, with_items, with_dict

The Simple Answer

Loops repeat a task for each item in a list. The modern syntax uses loop:. The older with_items: still works but loop: is preferred.

Common Patterns

List of packages: loop: [nginx, python3, git] with package: name={{ item }} state=present.

List of dictionaries: loop with complex data - each item has multiple fields: loop: [{name: alice, groups: admin}, {name: bob, groups: developer}] with user module.

with_dict: Iterate over a dictionary's key-value pairs.

with_fileglob: Iterate over files matching a pattern: with_fileglob: "files/*.conf" - copies all .conf files.

> **loop vs with_items:** loop is the modern, recommended syntax. with_items is legacy but still commonly seen in existing playbooks. Functionally similar for simple lists. loop supports more complex data structures and control keywords (loop_control for custom variable naming, index tracking).

Q12 Handlers and notify

The Simple Answer

Handlers are tasks that only execute when notified by another task that reported "changed." The most common pattern: copy a configuration file, notify a service restart handler. If the file did not change, the handler does not run - no unnecessary restart.

Key Behaviors

Handlers run at the END of the play, not immediately after the notifying task. If multiple tasks notify the same handler, it runs only once. Handlers run in the order they are defined, not the order they were notified. Force handler execution mid-play with meta: flush_handlers.

Common Mistakes

Name mismatch - The notify: value must exactly match the handler name. "Restart nginx" is not the same as "restart nginx" (case-sensitive). This is the #1 handler debugging issue. See scenario Q64.

Handler not running because task did not change - If the copy/template task reports "ok" (not "changed"), the handler is not triggered. This is correct behavior - no need to restart if the config did not change.

Q13 Templates - Jinja2

The Simple Answer

Templates are files with Jinja2 placeholders that Ansible fills in with variables before placing on the target. The template module processes the file on the control node and copies the rendered result to the managed node.

Jinja2 Syntax

Variables: {{ variable_name }} - replaced with the variable's value. **Conditionals:** {% if condition %}...{% endif %}. **Loops:** {% for item in list %}...{% endfor %}. **Filters:** {{ variable | default('fallback') }}, {{ list | join(',') }}, {{ string | upper }}.

Example

Template file nginx.conf.j2: server { listen {{ http_port }}; server_name {{ domain }}; root {{ app_root }}; }. Ansible renders it with the host's variables and places the result at /etc/nginx/conf.d/app.conf.

Templates vs copy: Use copy for files that are identical across all hosts. Use template for files that need per-host customization (different IPs, ports, domain names, feature flags).

Q14 Ansible Vault - encrypting secrets

The Simple Answer

Ansible Vault encrypts sensitive data (passwords, API keys, certificates) so they can be stored safely in Git alongside playbooks. Encrypted files look like random characters - unreadable without the Vault password.

Key Commands

ansible-vault create secrets.yml - Create a new encrypted file. **ansible-vault edit secrets.yml** - Edit an encrypted file (decrypts in memory, re-encrypts on save). **ansible-vault encrypt existing.yml** - Encrypt an existing file. **ansible-vault decrypt secrets.yml** - Decrypt permanently (be careful - file is now plaintext). **ansible-vault encrypt_string 'mypassword' --name 'db_password'** - Encrypt a single variable (embed in a regular YAML file).

Using Vault in Playbooks

ansible-playbook deploy.yml --ask-vault-pass - Prompt for password. ansible-playbook deploy.yml --vault-password-file=~/.vault_pass - Read from a file (used in CI/CD). The password file should be in .gitignore and have 600 permissions.

Best practice: Encrypt only the variables that need encryption, not entire files. Use ansible-vault encrypt_string to encrypt individual values within a regular vars file. This way you can see variable names in plain text and only the values are encrypted.

Q15 Roles - directory structure

The Simple Answer

Roles are the standard way to organize and reuse Ansible content. A role packages related tasks, handlers, templates, files, variables, and defaults into a self-contained unit.

Directory Structure

roles/nginx/ contains: tasks/main.yml (what to do), handlers/main.yml (service restart), templates/ (config files with Jinja2), files/ (static files to copy), vars/main.yml (role variables - high precedence), defaults/main.yml (default variables - lowest precedence, overridable), meta/main.yml (role dependencies and metadata).

Using Roles

In a playbook: roles: - nginx - applies the nginx role. Or in tasks: include_role: name=nginx - more control over when the role runs. Roles from Galaxy: install with ansible-galaxy install geerlingguy.nginx.

defaults vs vars: defaults/main.yml is meant to be overridden - it sets sensible defaults that users customize. vars/main.yml is NOT meant to be overridden - it sets internal role variables. Use defaults for configurable parameters (ports, paths, versions). Use vars for constants that should not change.

Q16 Galaxy - collections, requirements.yml

The Simple Answer

Ansible Galaxy is the community hub for sharing roles and collections. ansible-galaxy install role_name downloads and installs a role. Collections are the modern packaging format - they bundle roles, modules, plugins, and documentation in a namespaced package.

requirements.yml

List dependencies: roles and collections with version pinning. ansible-galaxy install -r requirements.yml installs everything. Always pin versions: version: 4.2.0 - unpinned dependencies can break unexpectedly. See scenario Q74.

Q17 Tags - running/skipping tasks

The Simple Answer

Tags label tasks so you can selectively run or skip them. ansible-playbook deploy.yml --tags "config" runs only tasks tagged "config." ansible-playbook deploy.yml --skip-tags "slow_tests" skips specific tasks.

Use Cases

Tag deployment tasks separately from configuration tasks. Run only "security" tagged tasks for a security patch. Skip "monitoring" tasks during emergency deploys. Tags enable a single comprehensive playbook to serve multiple purposes without duplication.

Q18 Idempotency - why it matters

The Simple Answer

Idempotency means running a playbook twice produces the same result as running it once. The second run should report "ok" (no changes) for every task. If a package is already installed, Ansible does not try to install it again. If a file already has the correct content, Ansible does not rewrite it.

Why It Matters

You can run playbooks as often as you want without fear. Cron-scheduled runs enforce desired state continuously. Partial failures can be recovered by re-running. Drift detection becomes free - if everything reports "ok," the system matches the desired state. If tasks report "changed," something drifted.

Non-Idempotent Modules

command and **shell** - Always report "changed" unless you use creates: or changed_when:. **raw** - For bootstrapping. No idempotency guarantees. These modules should be rare. If you find yourself using command/shell frequently, you are probably missing a purpose-built module.

> **The interview test:** "How do you know your playbook is idempotent?" Run it twice. The second run should show 0 changed tasks. If any task reports "changed" on the second run, it is not idempotent and needs fixing.

Q19 Connection plugins - SSH, local, docker, WinRM

The Simple Answer

Connection plugins define HOW Ansible connects to managed nodes.

SSH (default) - Standard for Linux/Unix. Uses OpenSSH. Fast with pipelining enabled. **paramiko** - Python SSH implementation. Fallback when OpenSSH is unavailable. Slower. **local** - Run on the control node itself. Used for localhost tasks (API calls, local file operations). **docker** - Connect to Docker containers. Runs modules inside

containers. **WinRM** - Connect to Windows hosts. Uses Windows Remote Management protocol. Requires pywinrm Python library on the control node. **network_cli** - Connect to network devices (switches, routers) via SSH with special handling for CLI prompts.

Q20 Privilege escalation - become

The Simple Answer

Most server configuration requires root privileges. become tells Ansible to escalate privileges on the managed node. become: yes activates escalation. become_user: root specifies the target user (default root). become_method: sudo specifies the method (sudo, su, pbrun, pfexec, doas).

Configuration Levels

Set globally in ansible.cfg, per play, or per task. Task-level become overrides play-level. Not every task needs root - only elevate when necessary (least privilege).

The sudo Password

If sudo requires a password: --ask-become-pass prompts interactively, or ansible_become_password in the inventory (Vault-encrypted). For passwordless sudo (common on cloud instances): the default sudo user has NOPASSWD in sudoers - no password needed.

Q21 Check mode and diff mode

The Simple Answer

Check mode (--check) - Dry run. Shows what WOULD change without making changes. Like terraform plan. Not all modules support check mode - command/shell skip entirely. Use for: validating playbooks before production runs, showing changes to reviewers.

Diff mode (--diff) - Shows the before/after difference for files changed by template, copy, and lineinfile modules. Combined with check mode (--check --diff): shows what files would change and exactly what the differences are - without making any changes.

> **Production workflow:** Always run --check --diff first. Review the output. Then run without --check to apply. This two-step process prevents surprises. In CI/CD, the PR pipeline runs check mode and posts the diff as a comment for review.

Q22 Ansible vs Puppet vs Chef vs SaltStack

The Simple Answer

Feature	Ansible	Puppet	Chef	SaltStack
Architecture	Agentless, push	Agent, pull	Agent, pull	Agent, push+pull
Language	YAML	Puppet DSL	Ruby DSL	YAML
Learning curve	Low	Medium	High	Medium
Idempotency	Built-in (modules)	Built-in (resources)	Built-in (resources)	Built-in

Scalability	Good (with AWX)	Excellent	Excellent	Excellent
Community	Largest	Large	Declining	Small
Best for	Multi-purpose, cloud	Enterprise compliance	Legacy, complex infra	Event-driven

Why Ansible Dominates

Agentless architecture eliminates installation, maintenance, and security overhead on every managed node. YAML is simpler than Ruby (Chef) or Puppet DSL. Multi-purpose - configuration management, application deployment, cloud provisioning, network automation, and orchestration all in one tool. The largest community and ecosystem.

> **The honest assessment:** Puppet is better for continuous compliance enforcement (agent constantly checks state). Chef has more mature testing frameworks. SaltStack is faster for real-time event-driven operations. Ansible wins on simplicity, versatility, and adoption. For most DevOps teams, Ansible is the right choice.

SECTION 2 - Mid-Level Production Patterns (Q23-Q46)

Q23 Dynamic inventory - AWS, GCP, Azure, inventory plugins

The Simple Answer

Dynamic inventory queries cloud APIs in real-time to discover hosts instead of maintaining a static file. Servers are created and destroyed constantly in cloud environments - static inventory files become stale within minutes. Dynamic inventory always reflects the current state.

Inventory Plugins (Modern Approach)

AWS: amazon.aws.aws_ec2 plugin. Create aws_ec2.yml: plugin: amazon.aws.aws_ec2, regions: [us-east-1], keyed_groups: [{key: tags.Environment, prefix: env}]. This creates groups automatically from EC2 tags - instances tagged Environment=production appear in the env_production group.

GCP: google.cloud.gcp_compute plugin. Azure: azure.azcollection.azure_rm plugin. Each works similarly - query the cloud API, build groups from tags/labels/metadata.

Constructed Inventory

The constructed plugin creates groups based on Ansible facts and host variables. Combine with cloud plugins: group hosts by OS family AND environment tag - groups like env_production_debian emerge automatically. Powerful for targeting subsets of hosts precisely.

> **The migration from scripts to plugins:** Older Ansible used executable scripts (ec2.py) for dynamic inventory. Modern Ansible uses inventory plugins - YAML configuration files that are simpler, more maintainable, and better integrated. Always use plugins for new setups.

Q24 Collections - namespaces, installation

The Simple Answer

Collections are the modern packaging format for Ansible content. They bundle related modules, roles, plugins, and documentation under a namespace: community.general, amazon.aws, kubernetes.core. Collections replaced the monolithic "everything included" model of Ansible 2.9.

Using Collections

Install: ansible-galaxy collection install amazon.aws. Pin in requirements.yml: collections: [{name: amazon.aws, version: ">=5.0.0,<6.0.0"}]. Use in playbooks: amazon.aws.ec2_instance module. Or add to collections: in the play to use short names.

Why Collections Matter

Ansible Core is now lean - only essential modules. Everything else is in collections with independent release cycles. amazon.aws can release a new EC2 feature without waiting for an Ansible Core release. This decouples module development from the Ansible engine.

Q25 Tower / AWX - GUI, RBAC, scheduling, workflows

The Simple Answer

AWX is the open-source upstream of Ansible Tower (now Ansible Automation Platform Controller). It provides a web UI, REST API, RBAC, job scheduling, credential management, and workflow orchestration for Ansible.

Key Features

Job Templates - Define: playbook, inventory, credentials, variables, limit, tags. Click "Launch" or schedule. No CLI knowledge needed for operators.

RBAC - Control who can run which templates against which inventories. Developers run deploy jobs. Operators run maintenance jobs. Security team runs audit jobs. Nobody has direct SSH access.

Credentials - Store SSH keys, cloud credentials, Vault passwords securely. Users use credentials without seeing them - the secret never leaves AWX.

Workflows - Chain multiple job templates with conditional logic. Deploy app → Run smoke tests → If pass: notify success. If fail: rollback → notify failure. Visual workflow editor.

Inventories - Sync dynamic inventories on a schedule. Cloud inventory refreshes every 5 minutes automatically.

Notifications - Slack, email, webhook notifications on job success/failure. Integration with PagerDuty for failed jobs.

> **AWX vs Tower vs AAP Controller:** AWX = free, upstream, community-supported. Tower = old commercial name, now deprecated. AAP Controller = current commercial product with support, certified content, and enterprise features. For production: AWX if you can support it yourself, AAP Controller if you need vendor support.

Q26 Optimization - pipelining, mitogen, async, serial, forks

forks

Number of parallel connections. Default is 5 - only 5 hosts at a time. For 100 hosts, this means 20 batches. Increase to 20-50: forks = 50 in ansible.cfg. Limited by the control node's CPU and memory. More forks = faster execution on large inventories.

Pipelining

Reduces the number of SSH operations per task. Without pipelining: Ansible creates a temp file, copies the module, executes it, and deletes it - multiple SSH round-trips. With pipelining: Ansible pipes the module directly into the Python interpreter - one SSH operation. Enable: pipelining = True in ansible.cfg. Requirement: requiretty must NOT be set in sudoers on the target (it usually is not on modern systems).

Mitogen

A third-party plugin that dramatically speeds up Ansible (2-7x faster). Replaces Ansible's SSH and module execution layer with an efficient remote procedure call system. Eliminates temp file creation and reduces SSH overhead. Install: pip install mitogen. Configure: strategy_plugins and strategy = mitogen_linear in ansible.cfg.

serial

Controls how many hosts are updated at once in a play. serial: 5 processes 5 hosts, then the next 5. serial: "25%" processes 25% of hosts at a time. Essential for rolling deployments - you never want to update all web servers simultaneously.

async

Run long tasks without blocking. async: 3600 (timeout in seconds), poll: 0 (fire-and-forget) or poll: 10 (check every 10 seconds). Used for: software installations that take 20+ minutes, database migrations, large file downloads. Without async, a slow task blocks the entire play for every host.

Q27 Error handling - block/rescue/always

The Simple Answer

block/rescue/always provides try/catch/finally for Ansible. block: contains tasks that might fail. rescue: runs when a block task fails. always: runs regardless of success or failure (cleanup).

Practical Example

block: deploy new version, run smoke test. rescue: rollback to previous version, send failure notification. always: remove maintenance mode page, notify team of result. This pattern ensures the system recovers gracefully even when deployment fails.

Other Error Handling

ignore_errors: yes - Continue even if the task fails. Use sparingly - usually indicates a design problem. Better to handle errors explicitly with block/rescue.

any_errors_fatal: true - If any host fails, stop the entire play for all hosts immediately. Essential for rolling deployments - if the first batch fails, do not continue deploying broken code to the remaining hosts.

max_fail_percentage - Allow a percentage of hosts to fail before aborting. max_fail_percentage: 10 allows 10% of hosts to fail. Used with serial for rolling updates where some host failures are acceptable.

Q28 Include vs import - static vs dynamic

The Simple Answer

import_tasks / import_role - Static inclusion. Processed at playbook parse time. All tasks are visible in --list-tasks. Tags and conditions on the import apply to all imported tasks. Cannot use loops on imports.

include_tasks / include_role - Dynamic inclusion. Processed at runtime when the task is reached. Tasks are not visible in --list-tasks until execution. Tags on the include do not propagate to included tasks (unless explicitly configured). Can use loops and conditionals to include different files based on runtime data.

When to Use Which

import - When the included content is always needed and you want tags/conditions to propagate. Use for: standard role inclusion, always-executed task files.

include - When inclusion depends on runtime conditions. Use for: OS-specific task files (include tasks/debian.yml when debian), environment-specific configurations, conditionally loaded features.

> **The trap:** Using include_role in a loop and expecting variables to be scoped per iteration. Variables from include_role can leak between iterations. Use import_role when possible, and be explicit about variable scoping with include_role. See scenario Q73.

Q29 Delegation - delegate_to, run_once

delegate_to

Execute a task on a different host than the play's target. The play targets webservers, but a specific task runs on the load balancer to drain traffic before updating each web server. delegate_to: loadbalancer.example.com. Facts are still collected from the original target host - the task runs on the delegate but with the original host's variables.

local_action / delegate_to: localhost

Run a task on the control node. Common for: API calls (creating DNS records, updating monitoring), sending notifications, waiting for external conditions.

run_once

Execute a task only once, regardless of how many hosts are in the play. Used for: database migrations (run once, not once per web server), creating shared resources, sending a single notification.

Q30 Rolling deployments - serial, max_fail_percentage

The Simple Answer

Rolling deployments update hosts in batches, keeping some hosts serving traffic while others are being updated. This provides zero-downtime deployments.

The Pattern

serial: "25%" - Update 25% of hosts at a time. Pre-task: remove host from load balancer. Tasks: deploy new version, restart service, run health check. Post-task: add host back to load balancer. Wait for health check to pass. Next batch.

Failure Control

max_fail_percentage: 10 - If more than 10% of hosts fail, abort. This prevents deploying a broken version to the entire fleet. any_errors_fatal: true - Stop at the first failure. More conservative.

Health Checks

After deploying to a batch, verify the application is healthy before proceeding. Use: uri module to check HTTP endpoints, wait_for module to verify ports are open, command module to run custom health check scripts. Only proceed to the next batch if health checks pass.

Q31 Magic variables - inventory_hostname, hostvars, groups

The Simple Answer

Magic variables are automatically set by Ansible and provide information about the current execution context.

inventory_hostname - The hostname as defined in the inventory (not the DNS name). Used for: accessing host-specific variables, identifying the current host in templates.

hostvars - Dictionary of all variables for all hosts. Access another host's variables: hostvars['db-server'].ansible_host. Used for: configuring a web server with the database server's IP address.

groups - Dictionary of all inventory groups and their members. groups['webservers'] returns a list of hostnames. Used in templates: generate a list of all web servers for load balancer configuration.

play_hosts - List of hosts in the current play. **ansible_play_hosts_all** - All hosts in the play (including failed). **inventory_dir** - Directory of the inventory file. **role_path** - Path to the current role.

Q32 Strategy plugins - linear, free, debug

The Simple Answer

Strategy plugins control the order of task execution across hosts.

linear (default) - Execute each task on all hosts before moving to the next task. All hosts complete task 1 before anyone starts task 2. Predictable but slow - fast hosts wait for slow hosts.

free - Each host proceeds through tasks independently at its own pace. Fast hosts finish quickly without waiting. Less predictable - tasks execute in different order on different hosts. Useful when tasks are independent and order does not matter between hosts.

host_pinned - Like free but limits the number of simultaneous hosts. More controlled parallelism.

debug - Interactive debugger. When a task fails, drops into a debugger where you can inspect variables, modify them, and retry the task. Invaluable for troubleshooting complex playbooks.

Q33 Ansible with Docker and Kubernetes

Docker

community.docker collection. Modules: docker_container (manage containers), docker_image (build/pull images), docker_compose (manage compose projects), docker_network, docker_volume. Connection plugin: community.docker.docker - run Ansible directly inside containers.

Kubernetes

kubernetes.core collection. Modules: k8s (create/update/delete any K8s resource from YAML), k8s_info (query resources), helm (manage Helm charts). Useful for: bootstrapping clusters, deploying applications, managing CRDs, and integrating K8s operations into larger workflows.

> **The decision:** Use Ansible for Kubernetes when you need to orchestrate K8s operations alongside non-K8s tasks (configure servers, update DNS, send notifications). Use kubectl/Helm directly when you are only managing Kubernetes resources. Do not use Ansible as a replacement for Helm - Helm handles templating and release management better for K8s-specific content.

Q34 Ansible in CI/CD - Jenkins, GitHub Actions, GitLab

The Pattern

CI/CD pipeline triggers Ansible playbooks for deployment. The pipeline provides: the version to deploy (extra_vars), the target environment (inventory selection), and credentials (from the CI platform's secret store).

GitHub Actions Example

Job step: uses ansible/ansible-action or runs ansible-playbook directly. Inventory and Vault password from GitHub Secrets. Deploy on merge to main. Staging deploys on PR merge. Production deploys with manual approval gate.

Best Practices

Never store SSH keys or Vault passwords in the repository. Use the CI platform's secret management. Pin Ansible version in the pipeline (requirements.txt or container image). Run ansible-lint before ansible-playbook. Run --check --diff in the PR pipeline for review. Apply with --diff in the deploy pipeline for audit.

Q35 Testing - Molecule, ansible-lint, yamllint

ansible-lint

Static analysis for playbooks. Catches: deprecated syntax, missing names on tasks, use of command/shell when a module exists, incorrect formatting, Vault password in plain text. Run in CI on every PR. Fix all warnings - they indicate real quality issues.

yamllint

YAML syntax validation. Catches: indentation errors, trailing spaces, line length violations. Complement to ansible-lint - catches structural YAML issues that ansible-lint misses.

Molecule

Integration testing for Ansible roles. Creates a fresh environment (Docker container, Vagrant VM, or cloud instance), applies the role, runs verification tests (Testinfra, Goss), and destroys the environment. Workflow: molecule create → molecule converge → molecule verify → molecule destroy. Each role should have Molecule tests in CI. Default scenario uses Docker - fast and free.

Testinfra

Python testing framework for infrastructure. Verify: a package is installed, a service is running, a file exists with correct permissions, a port is listening. Write tests in Python, Molecule runs them after applying the role.

Q36 Variable precedence - 22 levels

The Simple Answer

Ansible has 22 levels of variable precedence. The most important ones to remember (lowest to highest):

1. Role defaults (defaults/main.yml) - lowest, meant to be overridden.
2. Inventory file group_vars.
3. Inventory group_vars/ files.
4. Inventory file host_vars.
5. Inventory host_vars/ files.
6. Playbook group_vars/ files.
7. Playbook host_vars/ files.
8. Host facts (ansible_*).
9. Play vars.
10. Play vars_files.
11. Role vars (vars/main.yml).
12. Task vars.
13. set_fact / register.
14. Extra vars (-e) - highest, overrides everything.

The Practical Rule

> You only need to remember three levels: role defaults are lowest (designed to be overridden). Role vars are high (not meant to be overridden). Extra vars are highest (always win). Everything else falls in between in a mostly intuitive order. When confused, use ansible-inventory --host hostname to see the final resolved variables for a host.

Q37 Lookups and filters

Lookups

Read data from external sources. "{{ lookup('file', '/path/to/file') }}" - Read file content. "{{ lookup('env', 'HOME') }}" - Read environment variable. "{{ lookup('pipe', 'date +%Y%m%d') }}" - Run a command and capture output. "{{ lookup('password', '/dev/null length=20') }}" - Generate a random password. "{{ lookup('aws_ssm', '/prod/db_password') }}" - Read from AWS Parameter Store. Lookups run on the control node, not the managed node.

Filters

Transform data in Jinja2 expressions. "{{ list | sort }}" - Sort a list. "{{ string | hash('sha256') }}" - Hash a string. "{{ dict | dict2items }}" - Convert dict to list. "{{ variable | default('fallback') }}" - Default if undefined. "{{ variable | mandatory }}" - Fail if undefined. "{{ path | basename }}" - Extract filename from path. "{{ list | map('upper') | list }}" - Apply a filter to each list item.

Q38 Callback plugins

The Simple Answer

Callback plugins customize Ansible's output and behavior during execution. Built-in: default (standard output), json (machine-readable), yaml (cleaner output), timer (show task timing), profile_tasks (show per-task execution time - essential for performance optimization).

Enable: callbacks_enabled = timer, profile_tasks in ansible.cfg. profile_tasks shows which tasks take the longest - the first step in performance optimization.

Q39 Ansible and Terraform - when to use which

The Simple Answer

Terraform - Provision infrastructure (create VPCs, EC2 instances, RDS databases, Kubernetes clusters). Declarative. State file tracks resources. Excellent at creating and destroying cloud resources.

Ansible - Configure infrastructure (install packages, deploy applications, manage configs, restart services). Procedural tasks. No state file. Excellent at making servers ready to serve traffic.

Integration Pattern

Terraform creates the infrastructure → outputs IP addresses and inventory data → Ansible uses dynamic inventory or Terraform output to configure the created servers. Terraform provisions. Ansible configures. They complement, not compete.

> **The overlap:** Both can create cloud resources. Both can configure servers. But Terraform is better at infrastructure lifecycle (create, modify, destroy). Ansible is better at server-level configuration and multi-step orchestration. Use each for its strength.

Q40 Secrets management - Vault integration, no_log

HashiCorp Vault Integration

Lookup plugin: "{{ lookup('hashi_vault', 'secret=secret/data/myapp:db_password') }}". Ansible reads the secret from Vault at runtime. No secret stored in Ansible files. The Vault token or AppRole credentials must be available on the control node.

hashi_vault module: Ansible tasks that interact with Vault programmatically - write secrets, manage policies, enable auth methods.

no_log

no_log: true on a task prevents its input and output from appearing in logs. Essential when tasks handle passwords, tokens, or certificates. Without no_log, secrets appear in: terminal output, AWX job logs, callback plugin outputs, and CI/CD pipeline logs. See scenario Q65.

> **The layers of secret protection:** Vault encrypted files for secrets in Git. no_log on tasks handling secrets. Vault/Secrets Manager for runtime secret retrieval. Combine all three - each catches different exposure scenarios.

Q41 Multi-environment management

The Simple Answer

Maintain separate inventories for each environment: inventories/dev/hosts, inventories/staging/hosts, inventories/production/hosts. Each inventory has its own group_vars/ with environment-specific variables (database URLs, replica counts, feature flags).

The Command

ansible-playbook deploy.yml -i inventories/production/hosts - Run against production. ansible-playbook deploy.yml -i inventories/staging/hosts - Same playbook, different environment. The playbook is identical - only the inventory and its variables change.

Directory Structure

inventories/production/hosts (inventory file), inventories/production/group_vars/all.yml (production variables), inventories/production/group_vars/webservers.yml (production web server variables), inventories/production/host_vars/ (per-host overrides).

> **The safety mechanism:** Add a confirmation prompt for production: pre_tasks with a pause: prompt "You are deploying to PRODUCTION. Type 'yes' to continue." Or better: restrict production deployments to AWX/Tower with approval workflows and RBAC.

Q42 Windows management - WinRM

The Simple Answer

Ansible manages Windows hosts using WinRM (Windows Remote Management) instead of SSH. The control node must have pywinrm installed. Windows modules use PowerShell under the hood. Module names are prefixed with win_: win_service, win_package, win_file, win_copy, win_user, win_feature, win_reboot.

Setup

Enable WinRM on the Windows host (ConfigureRemotingForAnsible.ps1 script). Set ansible_connection: winrm, ansible_winrm_transport: ntlm (or kerberos for domain environments), ansible_port: 5986 (HTTPS) in inventory.

Q43 Network automation

The Simple Answer

Ansible manages network devices (Cisco, Arista, Juniper, etc.) using specialized connection plugins and modules. Connection: ansible_connection: network_cli (SSH to device CLI) or ansible_connection: netconf (NETCONF/YANG for modern devices).

Key Modules

ios_config - Cisco IOS configuration. **nxos_config** - Cisco Nexus. **eos_config** - Arista EOS. **junos_config** - Juniper Junos. **cli_command** - Generic CLI command execution.

Network-Specific Considerations

No Python on network devices - Ansible runs modules on the control node and sends commands over SSH/NETCONF. gather_facts: false (network devices do not support standard fact gathering - use ios_facts, eos_facts instead). backup: yes on config modules to save current config before changes.

Q44 Custom modules - writing Python modules

The Simple Answer

When no existing module fits your needs, write a custom module in Python. Place it in library/ directory alongside your playbook, or in a collection.

Module Structure

A Python script using AnsibleModule from ansible.module_utils.basic. Define argument_spec (expected parameters). Execute the desired operation. Return results with module.exit_json(changed=True, result=data) or module.fail_json(msg="error message").

When to Write Custom Modules

When you interact with an internal API not covered by any existing module. When you need complex logic that is awkward in Ansible tasks. When you need better idempotency than command/shell can provide. Most teams never need custom modules - the 3000+ existing modules cover most use cases.

Q45 Performance at scale - SSH multiplexing, fact caching

SSH ControlPersist

Reuses SSH connections across tasks. Instead of establishing a new SSH connection for each task (TCP handshake + TLS + authentication = 200-500ms), ControlPersist keeps the connection open and reuses it. Configure: ssh_args = -o ControlMaster=auto -o ControlPersist=60s in ansible.cfg. Default is already enabled in modern Ansible.

Fact Caching

Gathering facts (the setup module) on 500 hosts takes 2-5 minutes. Cache facts between runs: fact_caching = jsonfile (or redis for multi-user environments). fact_caching_timeout = 86400 (24 hours). Subsequent runs skip fact gathering entirely - saving minutes on every execution.

Other Optimizations

Disable fact gathering when not needed: gather_facts: false. Use forks = 50 (increase parallel connections). Enable pipelining = True. Use mitogen strategy (2-7x speedup). Minimize use of command/shell (they are slower than purpose-built modules). Use free strategy when task order between hosts does not matter.

Q46 Asynchronous actions - async, poll

The Simple Answer

Long-running tasks block the play - a 30-minute package installation holds up everything. async runs the task in the background. poll controls how Ansible checks on it.

Fire and Wait

async: 3600 (maximum runtime in seconds), poll: 30 (check every 30 seconds). Ansible starts the task, checks every 30 seconds, and continues when the task completes or the timeout is reached.

Fire and Forget

async: 3600, poll: 0 - Start the task and immediately continue with the next task. Check status later with async_status module and the jid (job ID) from register. Use for: starting multiple long tasks in parallel across different hosts, then waiting for all of them.

Practical Example

Trigger database backups on 10 database servers simultaneously (async with poll: 0). Each backup runs independently. After launching all backups, use async_status in a loop to wait for all to complete. Total time: the duration of the slowest backup, not the sum of all backups.

SECTION 3 - Senior Architecture & Advanced (Q47-Q55)

Q47 Architecture at scale - 1000+ hosts, AWX clustering

The Simple Answer

Single-instance Ansible works for hundreds of hosts. At 1000+ hosts, you need: AWX/Controller clustering (multiple nodes processing jobs in parallel), smart inventories (dynamic grouping across multiple inventory sources), execution environments (containerized Ansible with consistent dependencies), and credential management at scale (credential types, external credential lookups).

AWX Clustering

Multiple AWX nodes behind a load balancer. Jobs are distributed across nodes. The database (PostgreSQL) is shared. If one node fails, others continue processing jobs. Capacity-based scheduling assigns jobs to nodes with available resources.

Inventory at Scale

Dynamic inventory plugins query cloud APIs. Constructed inventory creates logical groups from facts and tags. Smart inventories in AWX combine multiple inventory sources into unified views. Cache inventory results to avoid hammering cloud APIs on every playbook run.

Execution Patterns

Never run playbooks against all 1000 hosts at once. Use --limit and serial to control blast radius. Segment by: region (deploy to us-east-1 first, then eu-west-1), role (deploy to canary hosts first, then 10%, then 100%), and criticality (non-production first, production last).

Q48 Execution environments - containerized Ansible

The Simple Answer

Execution Environments (EEs) are container images containing Ansible, collections, Python dependencies, and system libraries - everything needed to run a playbook. They replace the fragile "install everything on the control node" approach.

Why EEs Matter

Without EEs: different playbooks need different Python library versions (boto3 for AWS, google-auth for GCP, pywinrm for Windows). Version conflicts are common. "It works on my laptop" is real. With EEs: each playbook runs in a container with exactly the right dependencies. Consistent across developer laptops, CI/CD, and AWX.

Building EEs

ansible-builder creates EE images from a definition file. execution-environment.yml specifies: base image, Ansible collections, Python packages, and system packages. ansible-builder build --tag my-ee:1.0 produces a container image. Push to a registry. AWX/Controller uses EEs for every job execution.

ansible-navigator

Replaces ansible-playbook for local development with EEs. ansible-navigator run deploy.yml --eei my-ee:1.0 runs the playbook inside the EE container. Consistent with how AWX runs it. Eliminates "works on my machine" entirely.

Q49 Ansible Automation Platform - controller, hub, mesh

The Simple Answer

AAP is Red Hat's commercial Ansible platform with multiple components:

Automation Controller (formerly Tower) - Web UI, REST API, RBAC, job scheduling, workflows, credential management. The central management plane.

Automation Hub - Private Galaxy. Host certified and validated content (collections). Control which collections teams can use. Curated content with Red Hat support.

Automation Mesh - Distributed execution layer. Hop nodes relay traffic across network boundaries (DMZs, firewalls). Execution nodes run playbooks close to managed infrastructure. Replaces the old isolated nodes concept. Enables managing hosts across data centers, clouds, and network segments without direct SSH from the controller to every host.

Event-Driven Ansible (EDA) - See Q50.

> **When AAP vs AWX:** AWX for teams with Ansible expertise who can self-support. AAP for enterprises needing: vendor support, certified content, compliance certifications (SOC 2, FedRAMP), and the automation mesh for complex network topologies.

Q50 Event-Driven Ansible (EDA) - rulebooks, event sources

The Simple Answer

EDA watches for events from external sources (webhooks, monitoring alerts, logs, cloud events) and automatically triggers Ansible playbooks in response. Instead of an engineer seeing an alert, logging in, and running a playbook - EDA does it in seconds.

Components

Event sources - Webhook listener, Kafka consumer, alertmanager receiver, AWS EventBridge, file watcher, CloudWatch events. **Rulebooks** - YAML files defining: which events to watch, what conditions to match, and what actions to take (run a playbook, run a module, print a debug message). **Decision engine** - Evaluates rules against incoming events.

Example

Event source: Alertmanager webhook. Rule: when event.alert.labels.alertname == "HighCPU" and event.alert.labels.severity == "critical" → run_playbook: restart_service.yml with extra_vars from the alert (hostname, service name). Result: a critical CPU alert automatically triggers a service restart - no human intervention.

> **EDA transforms Ansible from "run when I tell you" to "run when something happens."** This is the bridge between monitoring and remediation. Combined with proper guardrails (approval workflows for production, dry-run first), EDA significantly reduces mean time to recovery (MTTR).

Q51 GitOps with Ansible - pull-based, drift detection

The Simple Answer

Traditional Ansible is push-based - you run a playbook manually or via CI/CD. GitOps with Ansible makes Git the source of truth. Changes are committed to Git, and automation applies them to infrastructure.

Pull Mode (ansible-pull)

ansible-pull runs on the managed node itself. It clones a Git repository, runs a local playbook, and applies the configuration. Scheduled via cron or systemd timer. Each node pulls its own configuration periodically - like Puppet's pull model but using Ansible.

Drift Detection

Run playbooks in check mode (--check) on a schedule. If any task reports "changed," drift is detected - the server no longer matches the desired state. Alert on drift. Auto-remediate or create a ticket depending on severity.

The Pattern

Developers commit config changes to Git → PR review → merge to main → AWX webhook trigger → playbook runs against affected hosts → configuration matches Git. No manual intervention. Every change is auditable (Git history). Rollback = revert the Git commit.

Q52 Immutable infrastructure vs configuration management

The Simple Answer

Configuration management (Ansible) - Mutate existing servers. Install packages, update configs, restart services on running servers. The server accumulates state over time.

Immutable infrastructure - Never modify a running server. Build a new machine image (AMI/VM image) with all changes baked in. Deploy by replacing servers, not modifying them. Old servers are terminated.

When Ansible Is Wrong

If you are building AMIs with Packer and deploying via ASG/MIG with zero in-place changes, Ansible for ongoing server management adds complexity without value. Ansible IS useful for building the AMI itself (Packer + Ansible provisioner), but not for managing the running instances.

When Ansible Is Right

Servers that cannot be easily replaced (databases, stateful services). Environments where image-based deployment is not feasible (bare metal, legacy). Multi-step orchestration that goes beyond server configuration (load balancer updates, DNS changes, database migrations, notifications). Network device configuration.

> **The modern pattern:** Immutable for stateless services (containers, auto-scaling groups). Ansible for stateful services, network devices, and orchestration tasks that span multiple systems. Many organizations use both - Ansible builds the image, Terraform deploys it, and Ansible handles Day 2 operations that do not fit the immutable model.

Q53 Security hardening

Ansible Security Best Practices

Control node - Harden the control node itself. Restrict who can run playbooks. Audit playbook executions. Use AWX/Controller with RBAC instead of direct CLI access. Store SSH keys and Vault passwords securely.

Privilege escalation - Use become only when needed, not globally. Use become_user to escalate to specific service accounts, not always root. Audit become usage in playbooks.

SSH hardening - Use SSH keys only (no passwords). Ed25519 keys. Disable root login. Use a dedicated Ansible user with limited sudo. Consider SSH certificates for automatic key rotation.

Vault - Encrypt all secrets. Use --vault-id for multiple passwords (different passwords for dev/prod). Rotate Vault passwords periodically. Never commit Vault passwords to Git.

no_log - Mark tasks handling sensitive data. Prevent secrets from appearing in output, logs, and AWX job records.

Audit - Log every playbook execution with who, what, when, and the result. AWX provides this automatically. For CLI usage, use callback plugins to log to a central system.

Q54 Migration from Puppet/Chef - strategy

The Migration Strategy

Phase 1: Inventory and assess. List all Puppet modules/Chef cookbooks. Map each to an Ansible equivalent (existing role/collection or needs to be written). Identify critical vs low-risk configurations.

Phase 2: Parallel run. Install Ansible alongside Puppet/Chef. Start with new servers - configure with Ansible from day one. Existing servers still managed by Puppet/Chef.

Phase 3: Migrate incrementally. Convert one module/cookbook at a time, starting with the simplest and least critical. Test with Molecule. Deploy to staging. Verify. Deploy to production. Remove the corresponding Puppet module/Chef cookbook.

Phase 4: Decommission. After all configurations are migrated, remove Puppet/Chef agents. Decommission the Puppet master/Chef server.

Common Pitfalls

Underestimating the effort - A 200-module Puppet codebase takes 3-6 months to migrate. **Losing drift enforcement** - Puppet continuously enforces state (agent runs every 30 minutes). Ansible only enforces when run. Implement ansible-pull or scheduled AWX jobs to match Puppet's enforcement frequency. **Variable mapping** - Puppet's Hiera and Chef's data bags map to Ansible's group_vars/host_vars, but the hierarchy and precedence differ. Plan the variable structure carefully.

Q55 Infrastructure testing - InSpec, ServerSpec, Goss

The Simple Answer

After Ansible configures a server, how do you verify it is correct? Infrastructure testing tools check the actual state against expectations.

InSpec - Chef's compliance testing framework. Ruby DSL. Tests: is nginx installed? Is port 80 listening? Does /etc/nginx/nginx.conf contain the correct settings? Has CIS benchmark compliance checks built in. Commercial with free open-source version.

ServerSpec - Ruby-based. Similar to InSpec but older. Tests server state from an SSH connection.

Goss - YAML-based. Fast. Lightweight Go binary - no Ruby dependency. Tests: packages, services, files, ports, users, commands. Runs locally on the target or via dgoss for Docker containers. Integrates with Molecule as a verifier.

Integration with Ansible

Molecule runs the role → then runs Goss/InSpec/Testinfra as verification. The test suite validates that Ansible achieved the desired state. Run in CI on every PR that modifies Ansible code. This catches: broken playbooks, missing handlers, incorrect template rendering, and configuration regressions.

SECTION 4 - Tricky Scenarios (Q56-Q75)

Q56 Playbook works on 1 host fails on 50

Step-by-Step

Step 1 - Is it SSH? ansible all -m ping. If some hosts fail, check: SSH keys distributed to all hosts, correct SSH user, sshd running, firewall allowing port 22.

Step 2 - Is it Python? Different hosts may have different Python versions or no Python at all. Check: ansible all -m raw -a "which python3". If Python is missing, use the raw module to install it: raw: apt-get install -y python3. See scenario Q69.

Step 3 - Is it OS differences? The playbook uses apt but some hosts are RHEL (need yum). Use the package module (auto-detects) or when: ansible_os_family conditionals.

Step 4 - Is it permissions? become works on one host but fails on others because sudo is configured differently. Check sudoers on failing hosts.

Step 5 - Is it forks? With forks: 5 (default), only 5 hosts are processed simultaneously. The error may only appear under parallel execution - race conditions in shared resources. Increase forks gradually and watch for patterns.

Step 6 - Run with -v or -vvv for detailed output on failing hosts. Limit to one failing host: --limit failing-host for focused debugging.

Q57 Task not idempotent - runs every time

Common Causes

shell/command without creates: A task using shell: pip install myapp always reports "changed." Fix: use the pip module (idempotent), or add creates: /usr/local/bin/myapp to skip if the binary already exists.

File ownership/permissions set by another process: A task sets file permissions to 644, but a running application changes them. On the next run, Ansible sees the difference and "changes" them back - every time.

Template with dynamic content: A template includes {{ ansible_date_time.iso8601 }} (current timestamp). Every run generates a different file - always "changed." Remove dynamic content from templates, or use changed_when: false if the change is expected and harmless.

How to Detect

Run the playbook twice. The second run should show 0 changed tasks. Any "changed" task on the second run is not idempotent. Fix each one.

Q58 Variable precedence - wrong value in production

What Happened

A database URL was defined in group_vars/all.yml as the production URL. But the staging inventory also uses group_vars/all.yml (from the playbook, not the inventory). The staging deployment connected to the production database.

Step-by-Step Debug

Step 1 - Find where the variable is defined. ansible-inventory -i inventory --host hostname --yaml shows all resolved variables for a host.

Step 2 - Check all variable sources. group_vars/all.yml at the inventory level AND the playbook level. Host-specific overrides. Extra vars passed on the command line.

Step 3 - Fix. Use inventory-specific group_vars: inventories/staging/group_vars/all.yml and inventories/production/group_vars/all.yml. Never put environment-specific variables in the playbook's group_vars - those are shared across all inventories.

Prevention

Keep environment-specific variables in the inventory's group_vars, not the playbook's. Use distinct variable naming: prod_db_url, staging_db_url is clearer than a single db_url that changes based on inventory context.

Q59 Vault password lost - secrets inaccessible

The Harsh Reality

If the Vault password is lost and there is no backup, the encrypted data is unrecoverable. Ansible Vault uses AES-256 encryption - brute-forcing is infeasible.

Step-by-Step Recovery

Step 1 - Search for the password. Check: CI/CD secrets, password managers (LastPass, 1Password, BitWarden), AWX/Tower credential store, environment variables on build servers, shell history on the control node (~/.bash_history - not recommended but people do this).

Step 2 - If the password is truly lost. The encrypted values must be recreated. Regenerate all secrets: new database passwords, new API keys, new certificates. Update the applications with new credentials. Create a new Vault-encrypted file with the new secrets and a new password.

Prevention

Store the Vault password in a dedicated password manager (not the same one that stores the encrypted values). Use --vault-password-file pointing to a script that reads from a secret manager (Vault, AWS Secrets Manager). Keep a recovery copy of the password in a physical safe (for disaster recovery). Use --vault-id with different passwords for different environments - losing the dev password does not affect production.

Q60 Playbook ran against wrong inventory - hit production

Step-by-Step Response

Step 1 - Assess damage. What did the playbook do? Check the output. If it deployed staging code to production - rollback immediately using the last known good version.

Step 2 - Rollback. If the playbook is idempotent and reversible: run the correct playbook against production. If it made irreversible changes: restore from backup. If it was a deployment: deploy the previous version.

Step 3 - Root cause. Was the default inventory pointing to production? Was there no confirmation step? Was the command history copy-pasted with the wrong inventory flag?

Prevention

Never set a default inventory to production. Require explicit -i inventories/production/hosts for production runs. Add a pre_tasks confirmation prompt for production: "You are targeting PRODUCTION. Type 'yes' to continue." Use AWX/Tower with RBAC - only authorized users can run jobs against production. Color-code terminals - red prompt for production context. Add --check --diff as the default first step.

Q61 Rolling deployment failed midway

What Happened

serial: 5 deployment. First batch (5 hosts) updated successfully. Second batch failed on 3 of 5 hosts. The play stopped due to max_fail_percentage. Now: 5 hosts on new version, 2 on new version (from the failed batch), 3 failed mid-update, and the remaining hosts on the old version.

Step-by-Step

Step 1 - Fix the failing hosts. Investigate why the second batch failed. Was it a deployment artifact? Missing dependency? Configuration issue?

Step 2 - Rollback the failed hosts. Run the playbook with the previous version, limited to the failed hosts: --limit @deploy.retry (the retry file contains failed hosts).

Step 3 - Decide: forward or backward. If the fix is simple, fix and continue the deployment. If the issue is complex, rollback all updated hosts to the previous version and investigate.

Step 4 - After fixing, complete the deployment. Resume from where it stopped: --limit 'all:!already_deployed_hosts' or simply re-run (idempotent tasks skip already-deployed hosts).

Prevention

> Always use max_fail_percentage or any_errors_fatal with serial. Include health checks after each batch. Use block/rescue for automatic rollback on failure. Test the deployment in staging with the same serial/batch settings.

Q62 Template rendered incorrectly - Jinja2 not expanded

Common Causes

Using {{ }} in a when statement. when: evaluates as Jinja2 already - do not wrap in {{ }}. Wrong: when: "{{ var }} == 'value'". Right: when: var == 'value'.

Missing variable. The variable used in the template is undefined. Ansible renders an empty string (or errors with undefined). Fix: define the variable or use {{ var | default('fallback') }}.

Raw block. The template contains {% raw %}...{% endraw %} around the section - intentionally preventing expansion. Check if this was added accidentally.

Wrong file extension. The template file is named .conf instead of .conf.j2. The template module processes both, but some editors or linters may not recognize it as a template.

Debugging

Use ansible-playbook --check --diff to see the rendered output. Or: ansible localhost -m template -a "src=template.j2 dest=/tmp/rendered.conf" to render locally and inspect the output.

Q63 Ansible taking 30 minutes for 100 hosts - optimize

Step-by-Step

Step 1 - Profile. Enable profile_tasks callback: callback_whitelist = profile_tasks in ansible.cfg. Re-run and identify which tasks are slowest.

Step 2 - Increase forks. Default is 5 - increase to 20 or 50: forks = 50. Immediate improvement for I/O-bound tasks.

Step 3 - Enable pipelining. pipelining = True eliminates extra SSH round-trips per task. 30-50% speedup.

Step 4 - Disable unnecessary fact gathering. If you do not use facts, add gather_facts: false. Or cache facts with fact_caching = jsonfile. Saves 2-5 seconds per host.

Step 5 - Use free strategy. If task order between hosts does not matter, strategy: free allows each host to proceed independently. Fast hosts finish without waiting for slow ones.

Step 6 - Replace shell/command with modules. Native modules use persistent connections. shell/command creates new SSH sessions. Modules are faster and more efficient.

Step 7 - Consider Mitogen. Install the mitogen strategy plugin. 2-7x speedup by replacing Ansible's SSH and execution layer. Dramatically reduces the number of SSH operations.

Step 8 - Batch large operations. Use async for long-running tasks. Combine multiple package installations into one task: package: name: [pkg1, pkg2, pkg3] state: present - one operation instead of three.

Q64 Handler not running - notify name mismatch

What Happened

A template task copies an nginx config file and notifies "Restart Nginx." The handler is named "restart nginx." The handler never runs because the names do not match - case-sensitive string comparison.

Debugging

Run with -v - Ansible shows "notifying handler Restart Nginx." Check handlers: the handler is named "restart nginx." The mismatch is the capital R and N.

Fix

Match the names exactly. Better: use a listen directive. Handler: listen: "restart web server." Notify: notify: "restart web server." This decouples the notification from the specific handler name and allows multiple handlers to respond to the same event.

Prevention

> Use ansible-lint - it catches handler name mismatches. Standardize handler naming: lowercase, descriptive. Use listen for flexibility. Test handlers in Molecule - verify that config changes trigger the expected service restarts.

Q65 Secret leaked in output - no_log missing

What Happened

A task retrieves a database password from Vault and passes it to a module. The task output shows the password in plain text in the terminal, CI logs, and AWX job log.

Step-by-Step

Step 1 - Add no_log: true to the task. This suppresses the task's input and output from all logs.

Step 2 - Clean up existing logs. Delete CI pipeline logs containing the secret. Clear AWX job output (Admin → Jobs → delete the job record). Purge terminal scrollback and shell history.

Step 3 - Rotate the secret. The password was exposed in logs - treat it as compromised. Change the database password and update all consumers.

Prevention

Add no_log: true to EVERY task that handles secrets. Use ansible-lint with custom rules that flag tasks using Vault variables without no_log. In AWX, use credential types that automatically mask values. Review CI pipeline logs regularly for accidental secret exposure.

Q66 Dynamic inventory returning stale hosts

What Happened

An EC2 instance was terminated. But the dynamic inventory still returns it. Ansible tries to connect and fails with SSH timeout - slowing the entire playbook by the connection timeout (30 seconds per stale host).

Causes

Caching - Inventory results are cached (fact_caching or inventory plugin cache_timeout). The cache has not expired yet. **AWS API lag** - EC2 API may take a few seconds to reflect termination. **Tag-based filtering too broad** - The inventory query matches instances in all states, including terminated (which persist in AWS for a while).

Fix

Reduce cache timeout. Add instance state filter in the inventory plugin: filters: instance-state-name: running - only return running instances. Set ansible_ssh_timeout to a shorter value (5 seconds) so stale hosts fail fast. Use --forks and async so one stale host does not block others.

Q67 Configuration drift - servers don't match playbook

Step-by-Step

Step 1 - Detect. Run ansible-playbook deploy.yml --check --diff. Any task reporting "changed" means the server has drifted from the desired state.

Step 2 - Investigate. What drifted? Check the diff output. Who changed it? Check audit logs on the server (auditd, auth.log). Was it a manual change or another automation tool?

Step 3 - Remediate. Run the playbook without --check to enforce the desired state.

Step 4 - Prevent. Schedule playbook runs via AWX/cron (every 30-60 minutes) to continuously enforce desired state. Restrict SSH access to prevent manual changes. Use immutable infrastructure where possible. Alert when --check detects drift (custom callback plugin or AWX notification).

The cultural fix: Establish a rule: never make manual changes to servers. All changes go through Ansible. If someone needs a quick fix, they commit the change to Git, the pipeline applies it. If it is truly urgent, run the Ansible playbook ad-hoc - still through automation, not manual SSH.

Q68 SSH host key checking failed on new servers

What Happened

ansible all -m ping fails with: "ERROR! host key verification failed." New cloud instances have unknown SSH host keys not in the control node's known_hosts.

Quick Fix

host_key_checking = False in ansible.cfg. Or ANSIBLE_HOST_KEY_CHECKING=False environment variable. This disables host key verification - the connection succeeds regardless of the host key. Fast but reduces security (vulnerable to MITM attacks).

Proper Fix

For cloud environments where instances are frequently created and destroyed, host key checking adds friction without much security value (the instances are ephemeral). Set host_key_checking = False in ansible.cfg for dynamic cloud environments. For static environments with long-lived servers, keep host key checking enabled and manage known_hosts with the known_hosts module or SSH certificates.

Best Practice

> Use SSH certificates (see Q8). The CA signs host keys automatically. The control node trusts the CA, not individual host keys. New instances get their host key signed by the CA during provisioning. No known_hosts management needed. Host key checking remains enabled with full security.

Q69 Cannot find Python on target host

What Happened

ansible all -m ping fails with: "MODULE FAILURE - No module named 'ansible.module_utils'." Or: "/usr/bin/python: not found."

Why It Happens

Ansible modules require Python on the target. Modern distributions install Python 3 but not always at /usr/bin/python. Some minimal images (containers, cloud images) have no Python at all.

Fix

Step 1 - Set the Python interpreter explicitly. ansible_python_interpreter: /usr/bin/python3 in inventory or group_vars. Auto-discovery: ansible_python_interpreter: auto (Ansible tries to find Python - usually works on modern Ansible).

Step 2 - If Python is not installed. Use the raw module (does not require Python): raw: apt-get install -y python3. Or use ansible_python_interpreter: auto_silent to suppress warnings.

Step 3 - For minimal container images. Install Python in the Dockerfile. Or use the raw module for bootstrapping.

Q70 Playbook hangs forever - SSH timeout

Common Causes

Interactive prompt. A task runs a command that prompts for input (password, confirmation). Ansible cannot interact - it hangs. Fix: pass input via stdin (echo 'yes' | command) or use module parameters that avoid prompts.

Firewall blocking SSH. The connection is attempted but packets are dropped (not rejected). SSH waits for the default timeout (30 seconds per host). Fix: check firewall rules. Reduce timeout: ansible_ssh_timeout = 10.

DNS resolution slow. Ansible resolves hostnames for each connection. Slow DNS adds seconds per host. Fix: use IP addresses in inventory, or fix DNS. Set UseDNS no in sshd_config.

ControlPersist connection stale. An old persistent SSH connection is broken but not cleaned up. Fix: remove stale control sockets from /tmp or ~/.ansible/cp/.

Debugging

Run with -vvvv (maximum verbosity). It shows the exact SSH command and where it is hanging. If it hangs on "Connecting to host," it is a network/firewall issue. If it hangs after connecting, it is a prompt or command issue.

Q71 become password required but not provided

What Happened

A task with become: yes fails: "Missing sudo password." The managed node's sudo requires a password, but Ansible was not given one.

Fix Options

Option 1 - Provide the password. --ask-become-pass (interactive prompt). Or ansible_become_password in inventory (Vault-encrypted). Or in AWX: Machine Credential with sudo password.

Option 2 - Configure passwordless sudo. Add the Ansible user to sudoers with NOPASSWD: ansible_user ALL=(ALL) NOPASSWD: ALL. This is the standard approach for cloud instances and automation users.

Option 3 - Use a different become_method. If sudo is locked down, try su, pbrun, or doas depending on the system.

> **Security note:** Passwordless sudo for the Ansible user is acceptable if: the user has no interactive login (shell set to /sbin/nologin), SSH key authentication is required, and the key is protected. The Ansible user should only be used by automation, never by humans.

Q72 File permissions changed on every run

What Happened

A file task sets mode: "0644" but reports "changed" on every run. The file never stays at the desired permissions.

Common Causes

Another process changes permissions. A cron job, application, or Puppet agent modifies the file after Ansible sets it. Fix: identify and stop the conflicting process. Or accept the drift and set changed_when: false if it is harmless.

umask interaction. When Ansible creates a file, the umask on the target may modify the final permissions. The copy and template modules set permissions explicitly after creation - this should not be an issue. But command/shell creating files ARE affected by umask.

SELinux context. The file module reports "changed" when the SELinux context does not match. The permission bits are the same, but the context differs. Fix: add sefcontext task to set the correct SELinux context.

String vs integer mode. mode: 644 (integer) is NOT the same as mode: "0644" (string). Integer 644 is octal 1204 - wrong permissions. Always quote file modes as strings: mode: "0644".

> **The golden rule:** Always quote file modes as strings in Ansible. mode: "0644" not mode: 0644. This is the #1 cause of "permissions change every run" bugs.

Q73 include_role variables leaking between hosts

What Happened

A playbook uses include_role in a loop for multiple services. Variables set inside the role for host A are visible when the role runs for host B - causing host B to get host A's configuration.

Why It Happens

include_role is dynamic - variables set with set_fact or register inside the included role persist in the host's variable scope. If the role runs on the same host for different loop iterations (or if variables are not properly scoped), values from the previous iteration leak.

Fix

Use import_role instead (static - better variable isolation). If include_role is required: explicitly reset variables at the start of the role. Use role defaults instead of set_fact for configuration. Scope variables with task-level vars instead of set_fact.

Q74 Galaxy role broke after update

What Happened

requirements.yml listed a Galaxy role without a version pin. ansible-galaxy install -r requirements.yml pulled the latest version. The new version had breaking changes - different variable names, removed tasks, changed defaults. The playbook failed.

Fix

Step 1 - Pin to the last working version. In requirements.yml: version: "4.2.0" (the version that was working before).

Step 2 - Re-install. ansible-galaxy install -r requirements.yml --force (force reinstall the pinned version).

Prevention

> Always pin role and collection versions. Use version ranges for safe updates: version: ">=4.0.0,<5.0.0" (any 4.x but not 5.0 which may have breaking changes). Review changelogs before updating major versions. Run Molecule tests after any dependency update. Use a lock file approach - test updates in CI before applying.

Q75 Write a playbook: deploy Nginx with zero downtime

The Design

Inventory: webservers group with 6 hosts behind a load balancer. **Strategy:** Rolling deployment with serial: 2 (2 hosts at a time).

Playbook Flow

Pre-tasks: Remove the host from the load balancer (API call or health check endpoint). Wait for connections to drain (pause: seconds=10 or wait_for module).

Tasks: Update Nginx package to the specified version. Deploy the new configuration template. Validate the config: command nginx -t (with changed_when: false). Notify the restart handler only if config changed.

Handlers: Restart Nginx (or reload if only config changed).

Post-tasks: Wait for Nginx to be healthy (uri module: check HTTP 200 on localhost). Add the host back to the load balancer. Wait for the load balancer to mark the host healthy.

Error Handling

any_errors_fatal: true - If one batch fails, stop immediately. Do not deploy broken config to more hosts. block/rescue: If deployment fails, rescue block restores the previous config and restarts Nginx. max_fail_percentage: 0 - Zero tolerance for failures.

Key Details

Use become: yes for package installation and config management. Use template for Nginx config (Jinja2 with server_name, upstream servers from inventory). Use --check --diff to preview changes before applying. Tag deployment tasks separately from config tasks. The config validation (nginx -t) runs before the restart - if the config is invalid, the handler is not notified and Nginx continues running with the old config.

What interviewers look for: Load balancer integration (drain before update, verify after update). Rolling strategy (serial, not all-at-once). Config validation before restart. Error handling (any_errors_fatal, block/rescue). Idempotency (running twice does nothing on the second run).

Interview Tips

For Junior Roles

Focus on Q1-Q22. Know playbook structure, common modules, variables, templates, and Vault. Be able to explain idempotency and why Ansible is agentless. Write a simple playbook on the spot.

For Mid-Level Roles

Q1-Q46. Be ready to discuss dynamic inventory, rolling deployments, error handling, Molecule testing, and variable precedence. Know the difference between include and import. Have a real story about a complex Ansible deployment.

For Senior Roles

All 75 questions. Execution environments, EDA, GitOps patterns, and scaling architecture separate senior from mid-level. Be ready to design an Ansible-based deployment pipeline and discuss when NOT to use Ansible (immutable infrastructure).

Quick Reference Cheat Sheet

Topic	Key Takeaway
Architecture	Agentless, SSH, push-based. Python required on targets. No agent to install/maintain.
Idempotency	Run twice = same result. Second run should show 0 changed. Test this explicitly.
Inventory	Static for small. Dynamic for cloud. Pin groups by tags/labels. Cache results.
Variables	Role defaults (lowest) → group_vars → host_vars → extra_vars (highest). Quote file modes.
Vault	Encrypt secrets. Use --vault-password-file for CI. Store password in password manager.
Roles	Reusable units. defaults/ for overridable params. vars/ for internal constants.
Handlers	Run at end of play. Only when notified by changed tasks. Case-sensitive names!
Templates	Jinja2. {{ var }} for values. {% if %} for conditionals. Always use .j2 extension.
include vs import	import = static, tags propagate. include = dynamic, runtime decisions.
Error handling	block/rescue/always. any_errors_fatal for rolling deployments. max_fail_percentage.
Performance	forks=50, pipelining=True, gather_facts: false, fact_caching, mitogen.
Rolling deploy	serial + max_fail_percentage + health checks + load balancer drain/restore.
Testing	Molecule + ansible-lint + yamllint in CI. Run --check --diff before applying.
AWX/Tower	RBAC, credentials, scheduling, workflows. Use for production. CLI for development.
Execution environments	Containerized Ansible. ansible-builder + ansible-navigator. Consistent dependencies.
EDA	Event-driven. Alertmanager webhook → Ansible playbook. Automated remediation.
Multi-env	Separate inventories per environment. Same playbook, different vars.
Secret safety	no_log: true on tasks with secrets. Vault for files. External lookups for runtime.
Drift detection	--check --diff on schedule. Any "changed" = drift. Alert and remediate.

Wrong inventory	Never default to production. Confirmation prompt. AWX RBAC. Color-coded terminals.